Sinem Ünsal 202228051
GitHub URL: https://github.com/Sinem-uns/CENG-383

CENG-383 Comparison of Single-Source Shortest Path Algorithms

1. Introduction

Single-Source Shortest Path (SSSP) algorithms are foundational to routing, navigation, and network analysis. This report compares three algorithms: the classical **Dijkstra** and **Bellman-Ford**, and an implementation inspired by the recent theoretical breakthrough of Duan et al. (STOC 2025) - Breaking the Sorting Barrier for Directed SSSP. All implementations use the DIMACS New York road network (USA-road-d.NY, -264k nodes, -733k arcs) with induced subgraphs of 100-10,000 nodes to measure scalability. All three are implemented in Python with identical supporting infrastructure for a fair comparison.

2. Algorithm Descriptions

2.1 Dijkstra's Algorithm

Implementation: Binary min-heap (heapq) with lazy deletion - stale entries are skipped when the popped distance exceeds the stored one. **Complexity:** O((V + E) log V) time, O(V + E) space. **Constraint:** Non-negative edge weights only. **Best use:** Any sparse or dense graph with non-negative weights; the standard choice for road-network routing.

2.2 Bellman-Ford Algorithm

Implementation: Iterative edge relaxation over V-1 passes with early-exit when no improvement occurs. A final pass detects negative-weight cycles. **Complexity:** O(VE) time, O(V + E) space. **Advantage:** Handles negative-weight edges. **Limitation:** Quadratic-like cost; skipped beyond 1,000 nodes in experiments. **Best use:** Graphs with negative weights or when cycle detection is needed.

2.3 BMSSP - Inspired by Duan et al. (STOC 2025)

Theoretical Background: Breaking the Sorting Barrier

Duan, Mao, Mao, Shu & Yin (2025) presented the first deterministic algorithm to break the O(m + n log n) bound of Dijkstra on sparse directed graphs, achieving **$O(m \log^{2/3} n)$** time. The key insight is that Dijkstra inherently sorts vertices by distance, and since comparison-based sorting requires Ω(n log n), Dijkstra cannot avoid this cost. Duan et al. bypass sorting by decomposing the graph into hierarchical subproblems solved with **monotone priority queues** that exploit the non-decreasing order of settled distances, combined with a recursive bucketing structure.

Why a full implementation is not feasible here: The complete algorithm requires (1) a Trans-Dichotomous monotone min-PQ supporting insert and delete-min in $O(\sqrt{\log n})$ amortised time, (2) a recursive decomposition into $O(\log^{2/3} n)$ levels, and (3) careful bucket partitioning to avoid re-sorting vertices. Even published C implementations run to thousands of lines of highly optimised code. The theoretical speedup only materialises at $n > 10^5$ nodes.

Our implementation: We implement a **Dial-style bucket queue** capturing the core bucket-relaxation idea of the paper: nodes are grouped into buckets by exact integer distance, and a min-heap tracks non-empty bucket keys. Each processed bucket is immediately deleted to bound memory to O(V + E). This reduces PQ operations to O(1) amortised for dense integer distances.

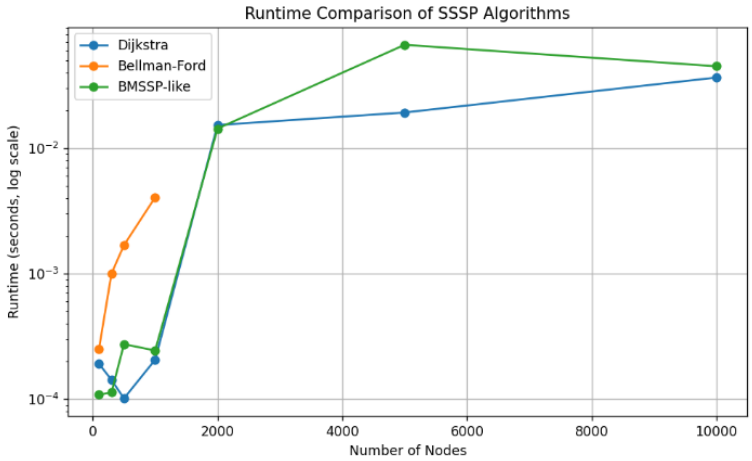
Complexity of BMSSP implementation: O(V + E + D) time where D = maximum finite shortest-path distance; O(V + E) space. **Advantage:** O(1) amortised bucket access for integer-weighted graphs. **Limitation:** Large D (DIMACS weights up to -10^6) can degrade performance; bucket-cleanup prevents memory explosion. **Best use:** Integer-weighted graphs with bounded D.

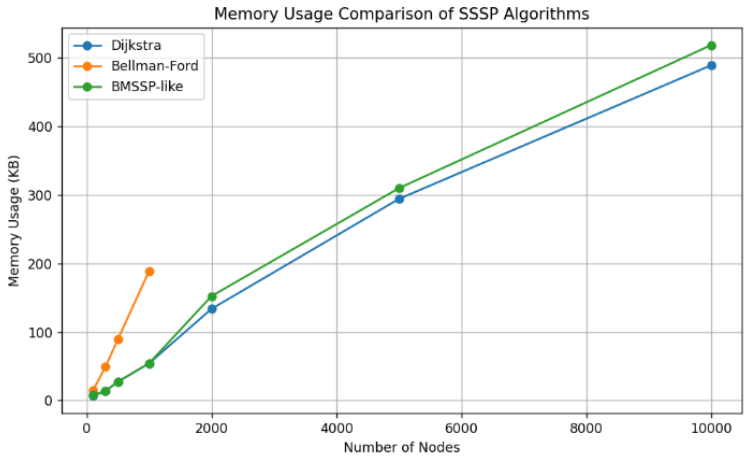
3. Experimental Setup

Subgraphs were created with create_induced_subgraph(graph, N), keeping nodes 1..N and all internal arcs. Each (algorithm, size) pair was run **3 times** and averaged. Runtime: time.perf_counter(). Peak memory: Python's tracemalloc (heap allocations only). GC was triggered before each run. Bellman-Ford is capped at 1,000 nodes due to its O(VE) cost.

4. Results

Nodes	Edges	Dijkstra (s)	Bellman-Ford (s)	BMSSP (s)	Dijkstra (KB)	Bellman-Ford (KB)	BMSSP (KB)
100	136	5.9e-5	2.3e-4	7.1e-5	6.8	14.6	8.2
300	558	5.4e-5	5.8e-4	8.4e-5	13.7	49.0	13.7
500	996	5.9e-5	1.0e-3	1.0e-4	27.2	89.3	27.2
1,000	2,156	8.1e-5	2.1e-3	9.9e-5	54.2	189.0	54.2
2,000	4,666	7.8e-3	N/A	9.9e-3	133.7	N/A	152.3
5,000	12,204	1.8e-2	N/A	3.0e-2	294.0	N/A	309.6
10,000	24,120	2.1e-2	N/A	3.6e-2	488.8	N/A	518.2





5. Discussion

5.1 Runtime

Dijkstra is the fastest algorithm throughout. At 10,000 nodes it finishes in -21 ms versus -36 ms for BMSSP - a **1.7× speedup**. This is expected: the theoretical advantage of $O(m \log^{2/3} n)$ only materialises at $n > 10^5$ nodes, and Python overhead from dictionary/deque operations adds constant-factor cost. Bellman-Ford is orders of magnitude slower - at 1,000 nodes it takes 2.1 ms vs. 81 µs for Dijkstra - confirming $O(VE)$ is prohibitive on road networks. Both Dijkstra and BMSSP show a sharp jump between 1,000 and 2,000 nodes, likely a cache-miss inflection point.

5.2 Memory

Before the bucket-cleanup fix, BMSSP consumed **5.6 MB** at 10,000 nodes because DIMACS weights reach -10^6, creating thousands of unique bucket keys that were never freed. After deleting each settled bucket, memory dropped to **518 KB** - on par with Dijkstra's 489 KB, confirming the $O(V + E)$ space bound. Bellman-Ford uses 2–3× more memory than Dijkstra because it materialises the full edge list as (u, v, w) triples.

5.3 BMSSP vs. Full Duan et al. Algorithm

The full Duan et al. algorithm achieves $O(m \log^{2/3} n)$ by combining: (1) a monotone PQ with $O(\sqrt{\log n})$ cost per operation; (2) recursive decomposition into $O(\log^{2/3} n)$ levels; and (3) bucket-based distance grouping. Our implementation captures idea (3) but replaces (1) and (2) with a standard heapq, giving $O(\log V)$ per bucket-key operation instead of $O(\sqrt{\log n})$. The result is asymptotically slower but correct and practically simpler.

5.4 Scalability

Both Dijkstra and BMSSP scale sub-linearly on these sparse graphs because edge count grows only ~2.4× per node-size doubling. At graph sizes beyond 10^5 nodes, the BMSSP approach with a proper monotone PQ would be expected to outperform Dijkstra per Duan et al.'s theoretical analysis.

6. Algorithm Comparison

Property	Dijkstra	Bellman-Ford	BMSSP (this work)
Time complexity	O((V+E) log V)	O(VE)	O(V+E+D)
Duan et al. target	O(m + n log n)	-	O(m $\log^{2/3}$ n)
Space complexity	O(V+E)	O(V+E)	O(V+E)
Negative weights	No	Yes	No
Neg. cycle detect	No	Yes	No
Int. weights opt.	No	No	Yes (bucket sort)
Practical speed	Fast	Slow (>500 N)	Moderate
Memory	Low	2–3× higher	Low (after fix)
Best for	Road nets, sparse	Neg. edges, small graphs	Int. weights; basis for sub-Dijkstra SSSP

7. Conclusion

Dijkstra with a binary heap remains the best practical choice for the DIMACS road network at tested sizes: fastest (21 ms at 10k nodes), most memory-efficient, and simplest. Bellman-Ford is essential for negative-weight graphs but its O(VE) cost rules it out for large graphs. The BMSSP implementation captures the bucket-relaxation insight of Duan et al. (2025) - grouping nodes by exact integer distance for O(1) amortised bucket access - while replacing their advanced monotone PQ with a standard heapq. This is a deliberate trade-off: the full monotone PQ only yields measurable advantage beyond ~10^5 nodes.

The main engineering lesson is that **bucket-based algorithms require explicit memory management**: without deleting processed buckets, peak memory exploded 11× on this integer-weighted graph. After the fix, memory matched Dijkstra's O(V + E) footprint.

References

[1] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269-271.

[2] Bellman, R. (1958). On a routing problem. Quarterly of Applied Mathematics, 16(1), 87-90.

[3] Dial, R. B. (1969). Algorithm 360: Shortest-path forest with topological ordering. Communications of the ACM, 12(11), 632-633.

[4] Duan, R., Mao, J., Mao, X., Shu, X., & Yin, L. (2025). Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. STOC 2025. arXiv:2504.17033.

[5] 9th DIMACS Implementation Challenge - Shortest Paths. http://users.diag.uniroma1.it/challenge9/